Swim Force

A Comprehensive Swimmer Tracking System

Cole Griscavage & Omar Ebied
Professor Zucker

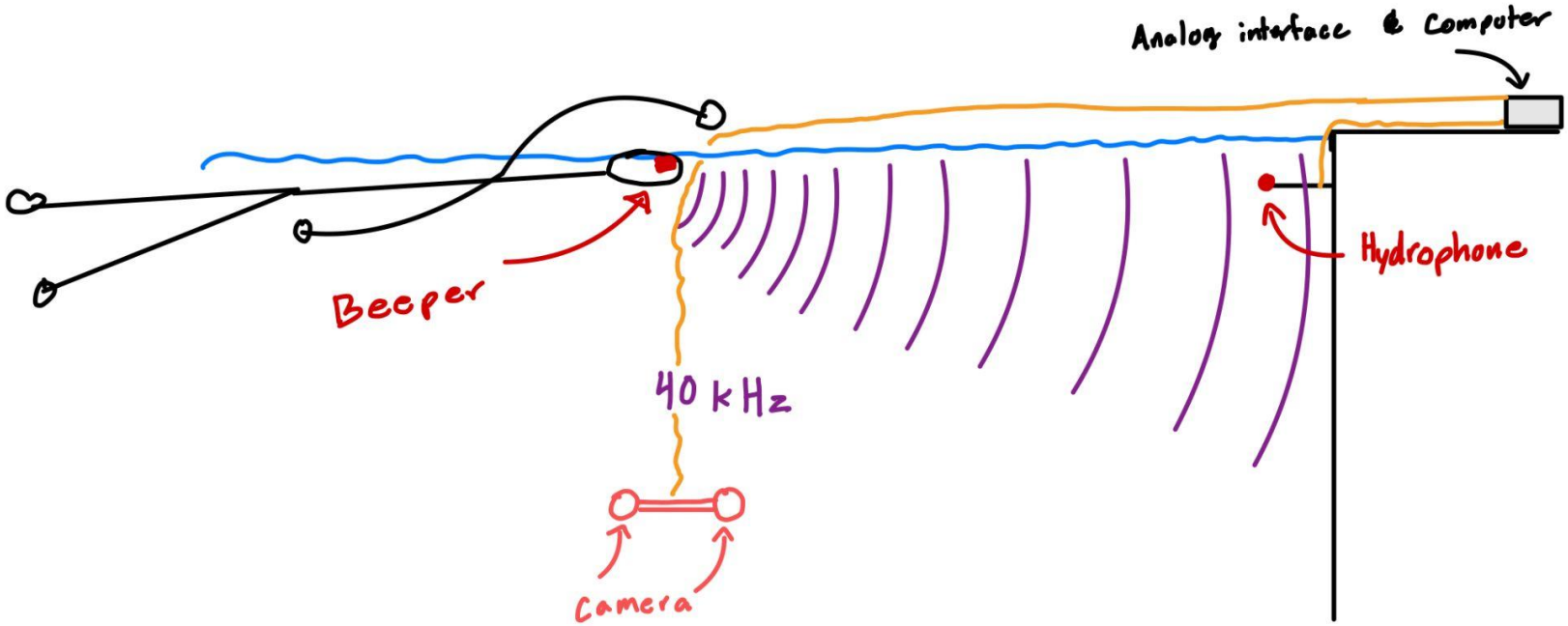
Introduction

- We proposed a comprehensive swimmer tracking system designed to improve performance analysis in swimming.
- Through the use of Ultrasonic transducers and underwater cameras, the system aims to provide precise data on swimmer velocity and enhance training feedback through automated footage sorting.
- By addressing inefficiencies in current training methods, SwimForce facilitates timely and personalized coaching for athletes.

01

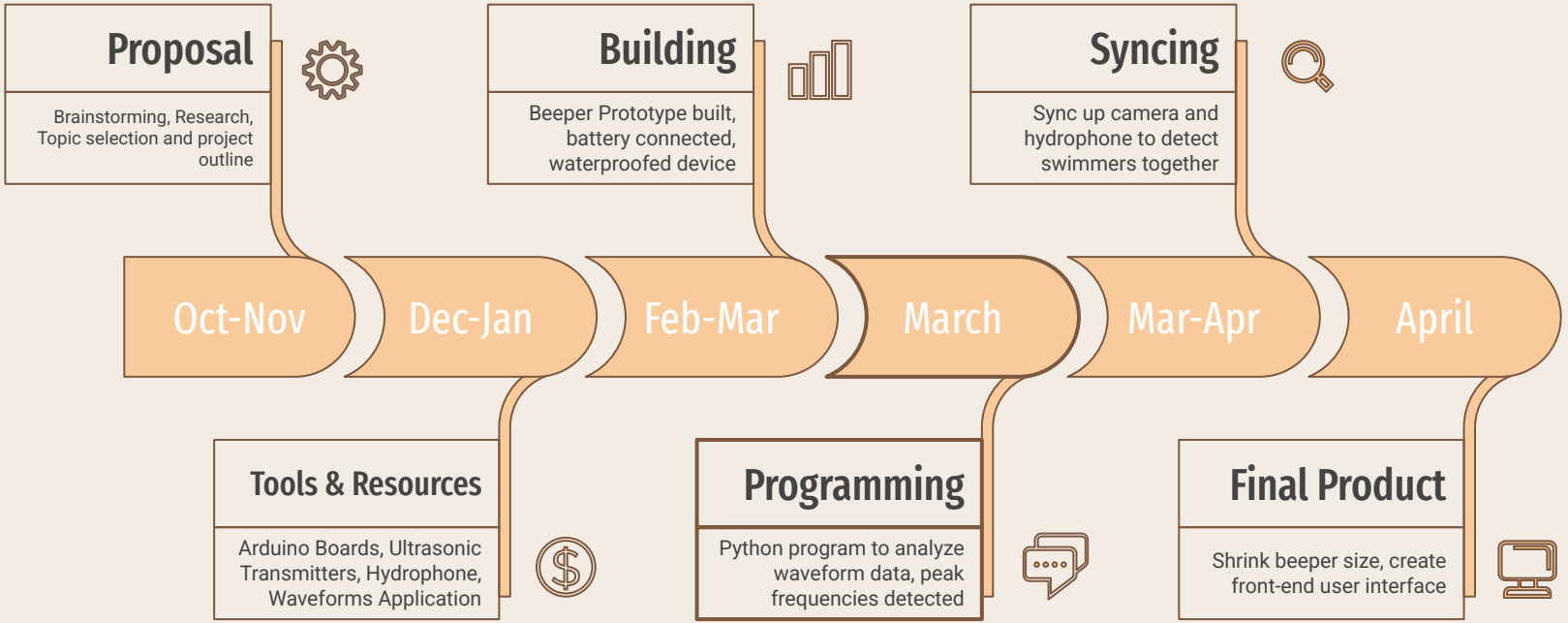
Goals & Overview

- Build ultrasonic transducer to emit customizable frequencies
- Waterproof the device & test with hydrophone in pool environment
- Use PoE cameras to capture clear footage look up from the bottom of the pool
- Write program to use hydrophone data to detect when a swimmer comes in range
- Edit program to sync camera input with hydrophone input to capture clips
- Use OpenCV to calculate swimmer velocity in acquired clips
- Edit program to collect and sort clips & data by athlete name


Analog interface & Computer
Beeper
Hydrophone
40 kHz
Camera

- [x] Build ultrasonic transducer to emit customizable frequencies
- [x] Waterproof the device & test with hydrophone in pool environment
- [x] Use PoE cameras to capture clear footage look up from the bottom of the pool
- [x] Write program to use hydrophone data to detect when a swimmer comes in range
- [ ] Edit program to sync camera input with hydrophone input to capture clips
- [ ] Use OpenCV to calculate swimmer velocity in acquired clips
- [ ] Edit program to collect and sort clips & data by athlete name

Timeline


Proposal
Brainstorming, Research, Topic selection and project outline
Building
Beeper Prototype built, battery connected, waterproofed device
Syncing
Sync up camera and hydrophone to detect swimmers together
Oct-Nov
Dec-Jan
Feb-Mar
March
Mar-Apr
April
Tools & Resources
Arduino Boards, Ultrasonic Transmitters, Hydrophone, Waveforms Application
Programming
Python program to analyze waveform data, peak frequencies detected
Final Product
Shrink beeper size, create front-end user interface

02

Technical Aspects

Programming Techniques

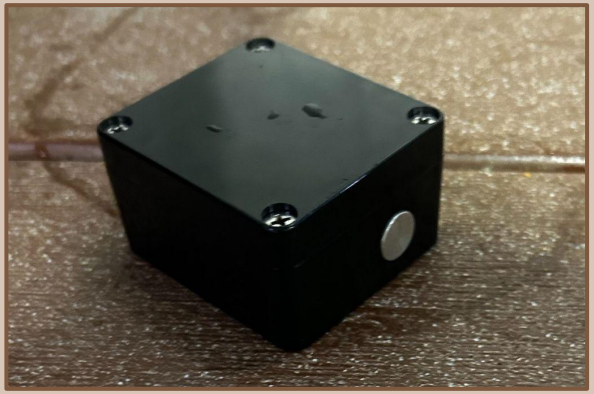
Python program to import the Analog Discovery 2 waveforms framework and collect audio through the hydrophone

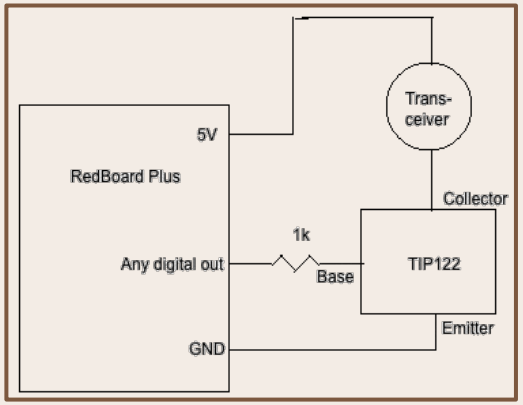
```python
import numpy as np
import time
import ctypes
import matplotlib.pyplot as plt
from scipy.signal import butter, lfilter

# === Load WaveForms SDK ===
dwf = ctypes.CDLL("/Library/Frameworks/dwf.framework/dwf")

# === Parameters ===
SAMPLE_RATE = 250_000  # 200 kHz
BUFFER_SIZE = 16384    # Match WaveForms settings
PULSE_DURATION = 10     # Monitor for 10 seconds
ORDER = 4  # Order of the bandpass filter
LOG_INTERVAL = 0.01  # Log data every 0.01 seconds
THRESHOLD_DBV = -70  # Threshold for logging timestamps

# Frequency ranges per person
PEOPLE = {
   # "cami": (19000, 21000),
   # "omar": (20100, 20300),
   "cole": (99000, 101000),
   # "ham": (49000, 51000),
}

# Track threshold crossings
swim_ins = {name: 0 for name in PEOPLE}
last_crossed = {name: False for name in PEOPLE}

# === Store data ===
timestamp_dbv_data = {name: [] for name in PEOPLE}
thresh_crossings = {name: [] for name in PEOPLE}

... # Device Initialization ...
```

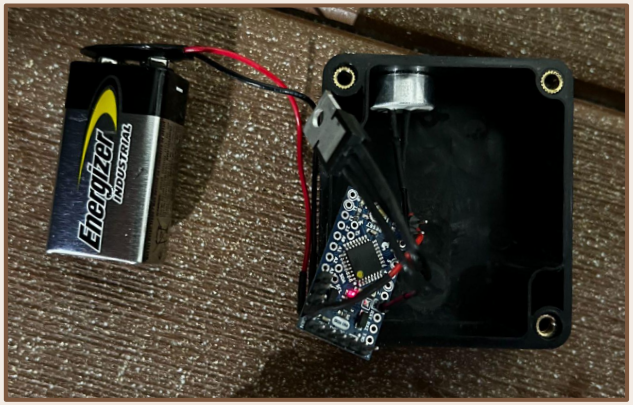
```python
# === Main Loop ===
hdwf = open_device()
if hdwf:
    configure_device(hdwf)
    start_time = time.time()
    last_log_time = start_time
    collecting = True

    while time.time() - start_time < PULSE_DURATION:
        current_time = time.time()

        if collecting and current_time - last_log_time >= LOG_INTERVAL:
            last_log_time = current_time
            collecting = False
        elif not collecting and current_time - last_log_time >= LOG_INTERVAL:
            last_log_time = current_time
            collecting = True

        if collecting:
            data = read_data(hdwf)

            for name, (low, high) in PEOPLE.items():
                filtered_data = butter_bandpass_filter(data, low, high, SAMPLE_RATE, order=ORDER)

                # Apply windowing and FFT
                window = np.blackman(len(filtered_data))
                windowed = filtered_data * window
                correction_factor = np.sum(window) / len(filtered_data)
                fft_result = np.fft.rfft(windowed) / (BUFFER_SIZE * correction_factor)
                magnitudes = 20 * np.log10(np.abs(fft_result) + 1e-12)

                freqs = np.fft.rfftfreq(len(filtered_data), 1 / SAMPLE_RATE)
                target_idx = np.where((freqs >= low) & (freqs <= high))[0]

                if len(target_idx) > 0:
                    peak_dbv = np.max(magnitudes[target_idx])
                    timestamp = current_time - start_time
                    timestamp_dbv_data[name].append((timestamp, peak_dbv))

                    if peak_dbv > THRESHOLD_DBV:
                        # Check for new threshold crossing
                        if not last_crossed[name]:
                            swim_ins[name] += 1
                            last_crossed[name] = True
                            print(f"{name} swam in at {timestamp:.2f} sec")
                            thresh_crossings[name].append(timestamp)
                    else:
                        last_crossed[name] = False
    close_device(hdwf)
```

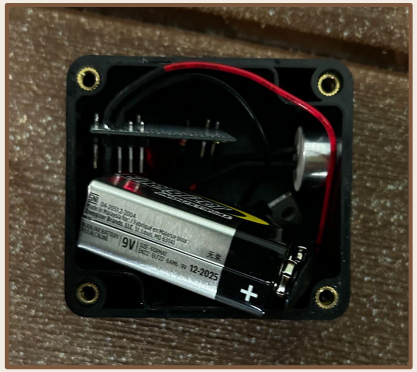
- Main loop containing FFT calculations to create spectrum from audio data
- Initialized, configured, and read data from Analog Discovery 2 prior to main loop
- Defined bandpass filter to hone in on specified frequencies
- Main loop collects data in intervals to reduce irrelevant data being read
- Use windowing before performing the FFT to improve accuracy of frequency analysis.
- Could get the dwf FFT analysis to work properly, had to resort to numpy
- Check for threshold crossing to detect if a swimmer came in range

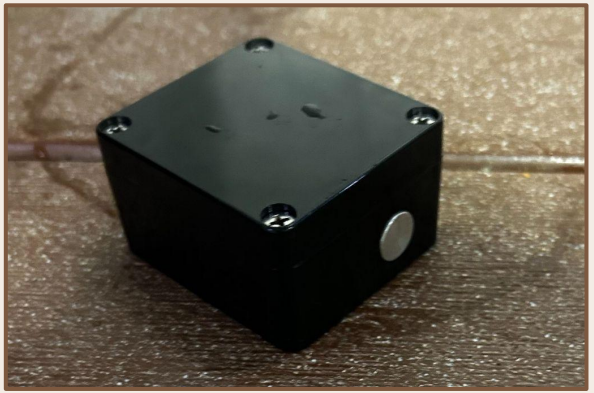
The Beeper



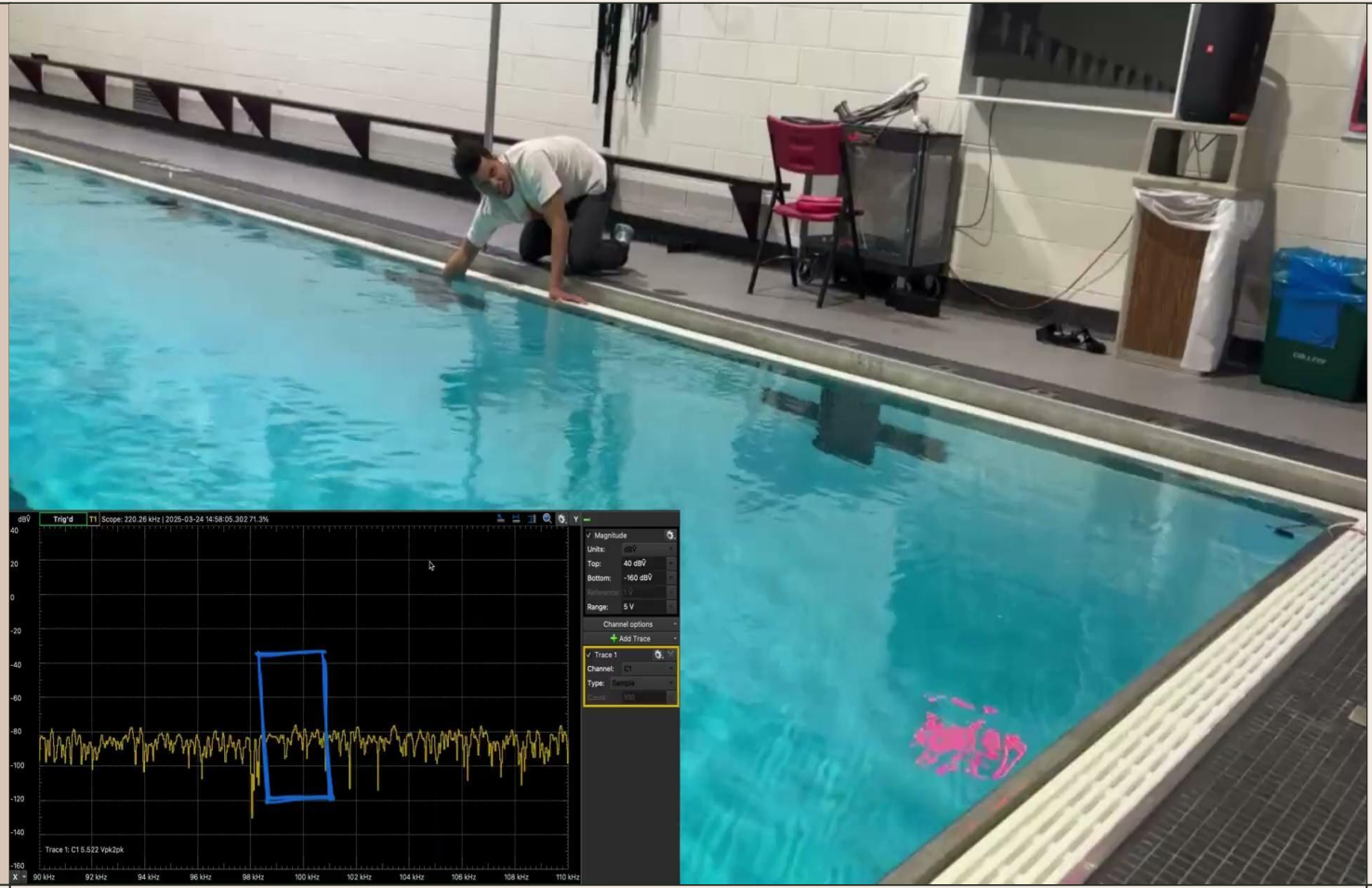

Trans-
ceiver
5V
RedBoard Plus
Collector
1k
Any digital out
Base
TIP122
Emitter
GND

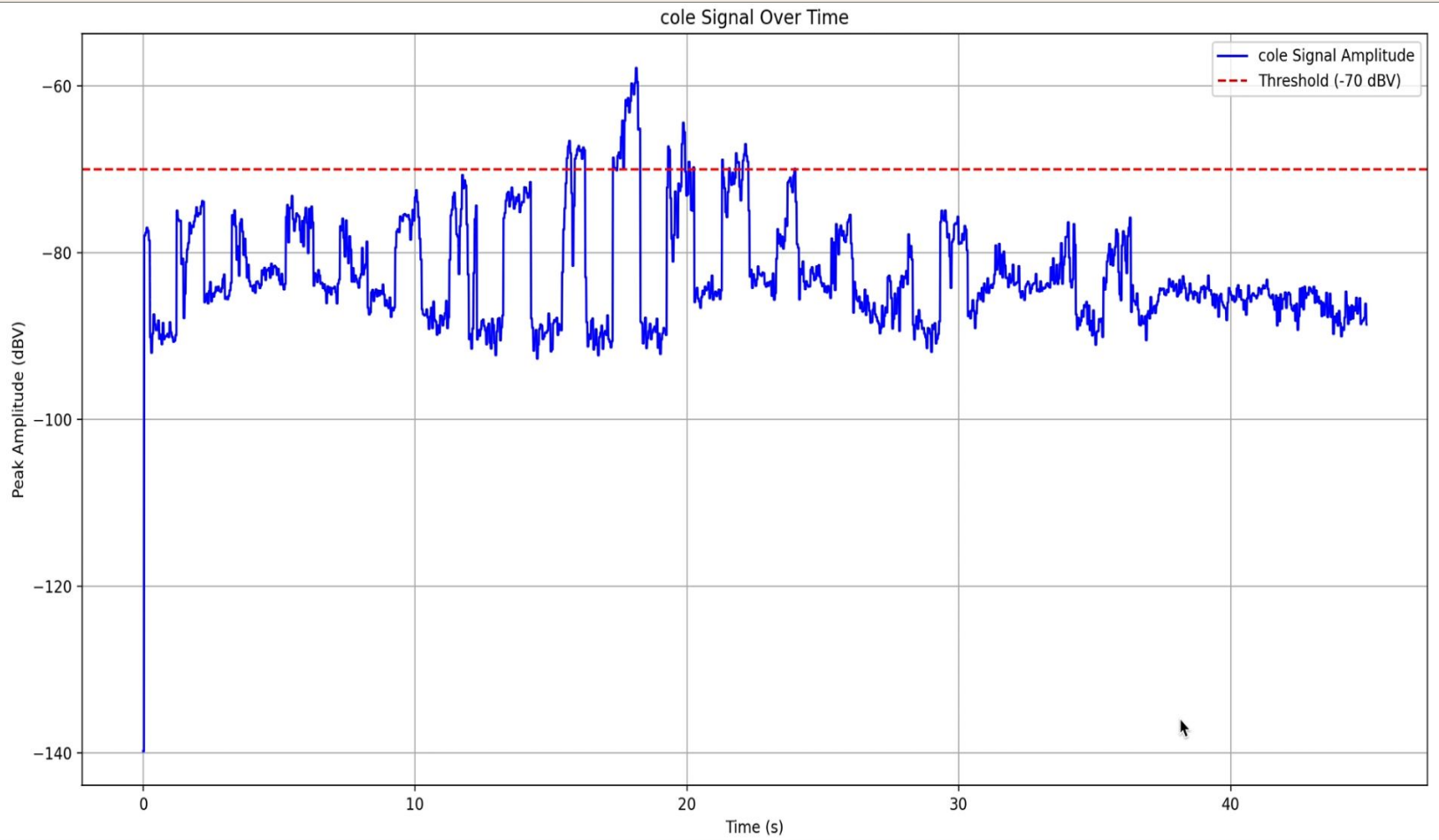

Energizer
INDUSTRIAL

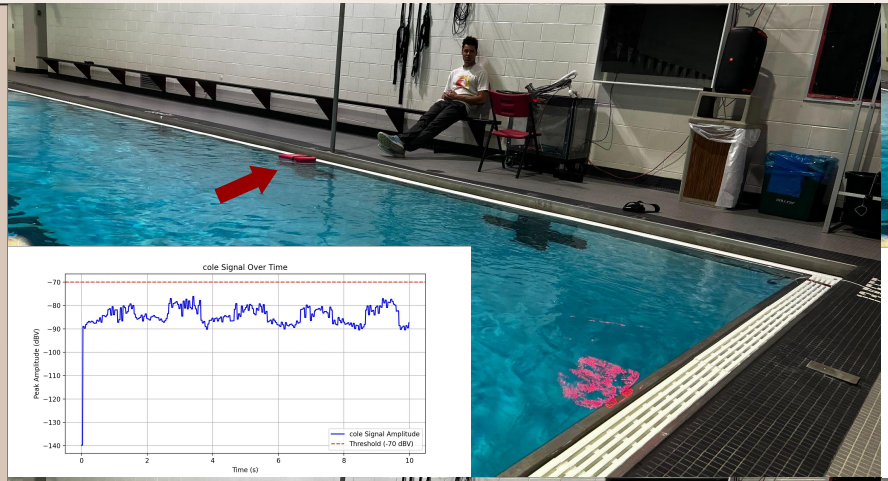


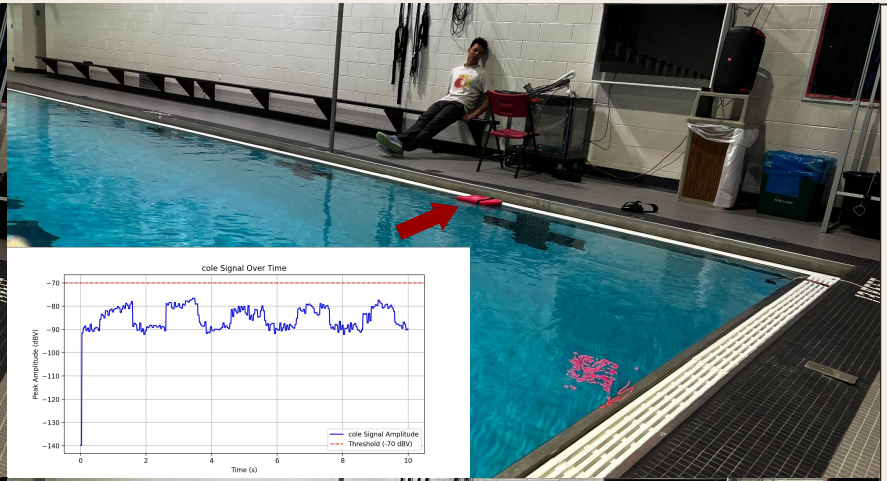


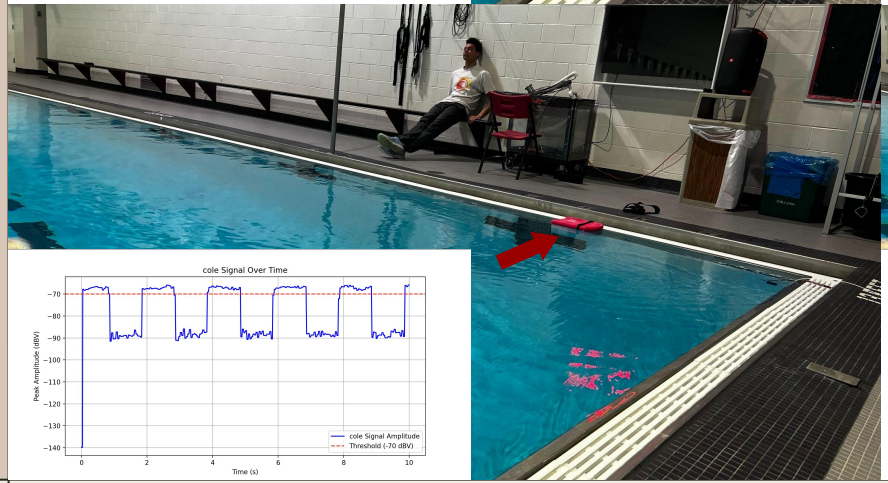
```
const int pin = 6;
const int time_on = 500;
const int time_off = 500;

void setup() {
  // put your setup code here, to run once:

}

void loop() {
  // put your main code here, to run repeatedly:
      tone(pin, 40000);  // Play tone at 9500 Hz
      delay(time_on);      // Wait for the duration specified in 'time'
      noTone(pin);     // Stop the tone
      delay(time_off);      // Wait for the same duration before repeating
}
```

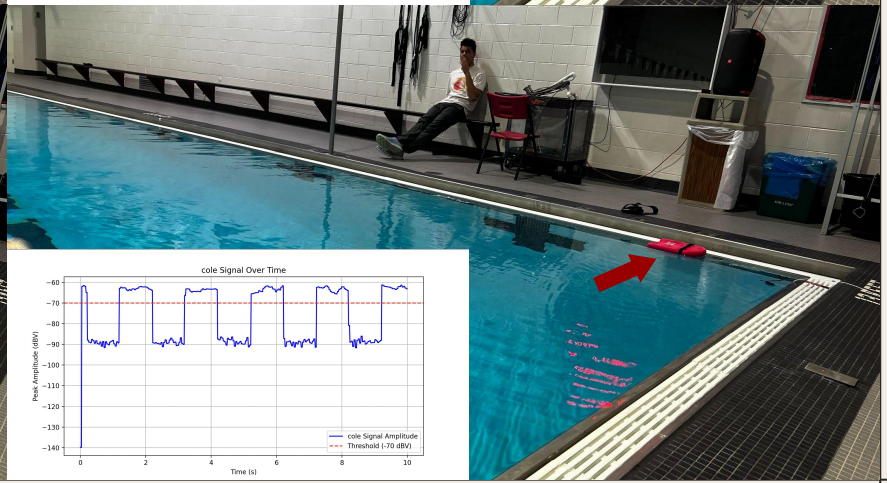

Trig'd
T1
Scope: 220.26 kHz | 2025-03-24 14:58:05.302 71.3%
dBV
40
20
0
-20
-40
-60
-80
-100
-120
-140
-160
Trace 1: C1 5.522 Vpk2pk
90 kHz
92 kHz
94 kHz
96 kHz
98 kHz
100 kHz
102 kHz
104 kHz
106 kHz
108 kHz
110 kHz
Magnitude
Units:
Top: 40 dBV
Bottom: -160 dBV
Range: 5 V
Channel options
Add Trace
Trace 1
Channel: C1
Type: Sample


cole Signal Over Time
cole Signal Amplitude
Threshold (-70 dBV)
Peak Amplitude (dBV)
−60
−80
−100
−120
−140
0
10
20
30
40
Time (s)

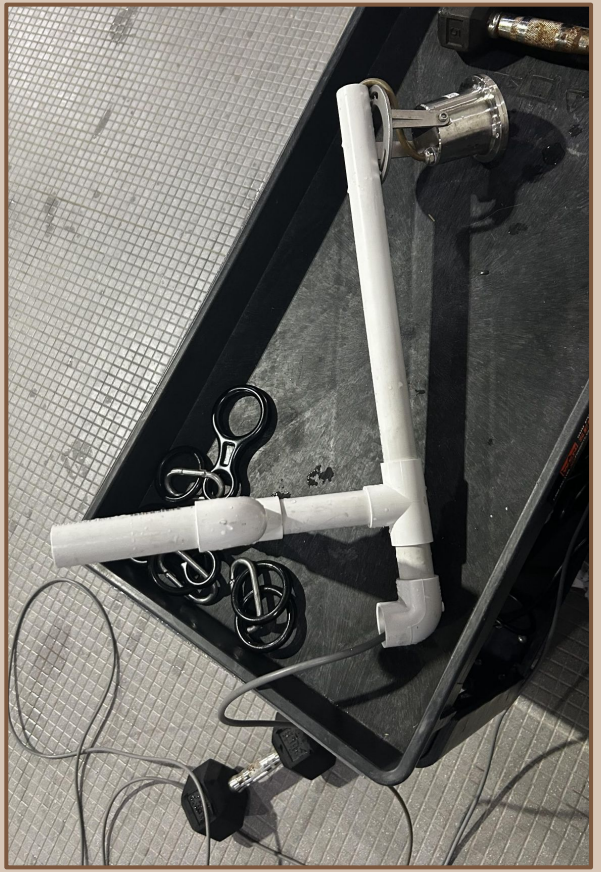

cole Signal Over Time
Peak Amplitude (dBV)
Time (s)
cole Signal Amplitude
Threshold (-70 dBV)


cole Signal Over Time
Peak Amplitude (dBV)
Time (s)
cole Signal Amplitude
Threshold (-70 dBV)


cole Signal Over Time
Peak Amplitude (dBV)
Time (s)
cole Signal Amplitude
Threshold (-70 dBV)


cole Signal Over Time
Peak Amplitude (dBV)
Time (s)
cole Signal Amplitude
Threshold (-70 dBV)

Video Footage / Object Tracking



Dual camera footage

Next Steps

- [x] Build ultrasonic transducer to emit customizable frequencies
- [x] Waterproof the device & test with hydrophone in pool environment
- [x] Use PoE cameras to capture clear footage look up from the bottom of the pool
- [x] Write program to use hydrophone data to detect when a swimmer comes in range
- [ ] **Edit program to sync camera input with hydrophone input to capture clips**
- [ ] **Use OpenCV to calculate swimmer velocity in acquired clips**
- [ ] **Edit program to collect and sort clips & data by athlete name**

Thank You!

Swim Force